

COCKPIT SIMULADOR COM FORCE FEEDBACK

Integração entre sistemas embarcados, eletrônica de potência, telemetria e controle de motores

ARDUINO LEONARDO

MKS ODRIVE + FFBEAST

SIMHUB + SOFTWARE PYTHON

Disciplina	Projeto de Sistemas da Computação
Docente	Prof. Dr. Jonathan de Matos
Projeto	Cockpit simulador com force feedback
Versão do documento	Documento técnico preliminar

Apresentação do documento

Este documento consolida a descrição técnica e acadêmica do projeto de construção de um cockpit simulador com force feedback. O conteúdo foi revisado para corrigir problemas gramaticais, remover repetições, uniformizar termos técnicos e alinhar as descrições aos componentes e códigos efetivamente utilizados no projeto.

ESCOPO

O sistema reúne um volante direct drive com motor BLDC de hoverboard, controladora MKS baseada em ODrive e firmware FFBeast; um módulo auxiliar com Arduino Leonardo; controles físicos; displays; telemetria pelo SimHub; e um software em Python que automatiza a seleção, compilação e gravação dos perfis do Arduino.

Sumário

1. Introdução e motivação
 2. Justificativa
 3. Objetivos
 4. Visão geral do sistema
 5. Controladora MKS ODrive e sistema FFBeast
 6. Motor BLDC de hoverboard
 7. Projeto de comunicação
 8. Componentes associados ao Arduino Leonardo
 9. Software desenvolvido para o cockpit
 10. Modelagem de dados
 11. Organização dos códigos
 12. Considerações de segurança e limitações
 13. Considerações finais
- Referências técnicas

1. Introdução e motivação

A idealização deste trabalho para a disciplina de Projeto de Sistemas da Computação, ministrada pelo Prof. Dr. Jonathan de Matos na Universidade Estadual de Ponta Grossa, representa mais do que uma atividade avaliativa: trata-se da continuidade de um interesse construído desde a infância por simuladores de corrida e tecnologias de interação.

Volantes comerciais como Logitech G27 e G29 sempre despertaram interesse por proporcionarem uma experiência mais próxima da condução real. Entretanto, durante muitos anos, a aquisição de um equipamento desse tipo não foi viável. Com o ingresso no curso de Engenharia de Computação, surgiu a oportunidade de compreender os princípios de eletrônica, programação e controle necessários para desenvolver uma solução própria.

O primeiro protótipo foi construído a partir de componentes reaproveitados de um volante de PlayStation 2, provavelmente comercializado para o jogo Gran Turismo 4. O dispositivo possuía dois pedais com potenciômetros e um sensor de direção. Após a desmontagem, esses componentes foram adaptados a um Arduino Leonardo, resultando em um volante simples, robusto e funcional para computador. A montagem tomou como referência conteúdos educacionais do canal Gustavo G-Tec, que apresenta projetos de volantes artesanais e sistemas básicos de force feedback com motores de corrente contínua e redução mecânica.

Esse primeiro volante foi apresentado como trabalho da disciplina de Sistemas Embarcados, ministrada pela professora Gabrielly de Queiroz Pereira. A experiência demonstrou que o projeto poderia evoluir para um sistema mais completo, integrando eletrônica de potência, controle de motor, software embarcado, comunicação com jogos, telemetria e construção mecânica.

Dessa forma, o cockpit atual foi proposto como uma evolução técnica e acadêmica do protótipo inicial. O projeto busca aplicar conhecimentos adquiridos no curso em um equipamento real e funcional, conciliando fundamentação teórica, desenvolvimento prático e lazer.

2. Justificativa

O projeto oferece a oportunidade de integrar software e hardware por meio da eletrônica de potência, da lógica de microcontroladores, da leitura de sensores e da comunicação entre dispositivos. Por esse motivo, sua proposta se adequa diretamente aos conteúdos trabalhados na disciplina de Projeto de Sistemas da Computação.

A construção do cockpit permite aplicar conhecimentos em um sistema real, envolvendo Arduino, sensores, controles físicos, displays, telemetria, programação para computador e um sistema de force feedback. Sua arquitetura modular também possibilita documentar, testar, substituir e aprimorar cada subsistema de forma organizada, atendendo aos objetivos práticos e técnicos da disciplina.

3. Objetivos

3.1 Objetivo geral

Desenvolver um cockpit simulador funcional, capaz de integrar comandos físicos, telemetria e um sistema de force feedback baseado em motor BLDC de hoverboard, controladora MKS ODrive e firmware FFBeast.

3.2 Objetivos específicos

- Implementar o sistema de force feedback com motor trifásico BLDC de hoverboard e encoder incremental para leitura da posição angular do volante.
- Construir um painel com botões, chave de seta, câmbio H-shifter, freio de mão analógico, display LCD de telemetria e display de sete segmentos para indicação da marcha.
- Desenvolver placas de interligação para o Arduino Leonardo e para o acionamento das ventoinhas de refrigeração.
- Integrar os pedais analógicos às entradas GPIO da controladora MKS por meio dos recursos do FFBeast Premium.
- Aprofundar habilidades em eletrônica de baixa potência, eletrônica de potência, soldagem, interpretação de diagramas e leitura de folhas de dados.
- Utilizar ferramentas de programação e automação, incluindo Arduino, JavaScript no SimHub, Python e Arduino CLI.
- Produzir documentação técnica, esquemas, modelos, códigos e manuais que permitam compreender, operar e futuramente aprimorar o sistema.
- Empregar o cockpit em demonstrações acadêmicas, como feiras de profissões, apresentando aplicações práticas e atrativas da Engenharia de Computação.

Para os pedais, foram utilizadas entradas analógicas disponíveis na controladora MKS. O acelerador foi adaptado a partir de um pedal eletrônico de Mitsubishi Outlander 2014, escolhido por possuir sensor interno adequado à leitura contínua da posição. A adoção desse componente facilitou a integração com o sistema, uma vez que seu funcionamento já é baseado em um sinal proporcional ao deslocamento do pedal.

CARÁTER DIDÁTICO

Além do resultado técnico, o projeto possui caráter demonstrativo. Ele evidencia como conhecimentos de microcontroladores, software, eletrônica de potência e modelagem podem ser combinados na construção de um equipamento interativo, útil para atividades acadêmicas e também voltado ao entretenimento.

4. Visão geral do sistema

O cockpit foi organizado em subsistemas com responsabilidades específicas. Essa divisão reduz a complexidade de cada módulo e facilita testes, manutenção e futuras expansões.

Subsistema	Principais elementos	Função
Force feedback	MKS ODrive, STM32, FFBeast, motor BLDC, encoder e resistor de frenagem	Produzir torque no volante e reproduzir os efeitos enviados pelo jogo.
Controles auxiliares	Arduino Leonardo, multiplexador 74HC4067, botões, câmbio e freio de mão	Ler comandos físicos e enviá-los ao computador como dispositivo HID.
Telemetria	SimHub, comunicação serial e LCD 20×4	Obter dados dos jogos, tratá-los e apresentá-los no painel.
Indicação de marcha	Arduino, CD4026 e display de sete segmentos	Apresentar a posição selecionada no câmbio.
Software de seleção	Aplicação Python e Arduino CLI	Selecionar o perfil do jogo, compilar o código e gravá-lo no Arduino Leonardo.
Refrigeração	Ventoinhas e placa de acionamento	Auxiliar na dissipação térmica da controladora, do resistor e do motor.

5. Controladora MKS ODrive e sistema FFBeast

Para o controle do force feedback foi escolhida uma controladora MKS baseada na arquitetura ODrive. Esse tipo de placa é apropriado para motores BLDC porque permite controlar corrente, posição, velocidade e torque de forma muito mais precisa do que um controlador convencional de velocidade.

O núcleo lógico da placa é um microcontrolador STM32. Ele executa o firmware, lê os sinais do encoder, estima a posição e a velocidade do eixo e calcula os comandos enviados à etapa de potência. Essa etapa utiliza drivers de gate e MOSFETs organizados em uma ponte inversora trifásica, responsável por aplicar as tensões necessárias às três fases do motor.

O controle do motor é realizado por FOC, sigla para Field-Oriented Control. Nesse método, as correntes do motor são tratadas em um sistema de referência associado ao campo magnético do rotor. Isso possibilita controlar diretamente a componente responsável pelo torque. Os MOSFETs são acionados por modulação PWM trifásica, normalmente associada à modulação vetorial espacial, e não por um único sinal PWM aplicado diretamente ao motor.

O encoder fecha a malha de controle ao informar continuamente a posição do eixo ao STM32. A controladora compara o estado medido com o torque solicitado e corrige a corrente das fases. Esse processo permite gerar respostas rápidas e suaves, inclusive com o volante em baixa velocidade ou praticamente parado.

A escolha da MKS ocorreu por sua compatibilidade com o FFBeast, disponibilidade, custo e capacidade de controlar motores de hoverboard. Em comparação com alternativas como placas ODESC e outras versões derivadas do ODrive, a opção escolhida apresentou uma relação adequada entre recursos, facilidade de integração e espaço físico. A comparação entre versões Mini, XDrive-S e modelos semelhantes deve considerar a revisão exata da placa, os MOSFETs empregados, os limites de tensão e corrente, os conectores e a capacidade de refrigeração.

O FFBeast é um firmware voltado à construção de volantes direct drive. Após sua gravação na controladora, a placa passa a ser reconhecida pelo computador como um dispositivo de entrada com suporte a force feedback. O jogo envia os efeitos de força; o firmware interpreta esses comandos, combina os efeitos ativos e os converte em uma referência de torque para o controle do motor.

A versão gratuita do FFBeast permite utilizar as funções essenciais do volante. A licença Premium libera recursos adicionais, incluindo o uso de eixos analógicos conectados diretamente às entradas da controladora. No projeto, esse recurso é utilizado para os pedais. O freio de mão, por sua vez, permanece conectado ao Arduino Leonardo como eixo HID independente.

A licença Premium é vinculada ao identificador único do STM32. Após a aquisição, uma chave correspondente ao dispositivo é inserida no software de configuração. A ativação ocorre pela comunicação USB com a controladora, não por uma porta serial convencional do Arduino.

GRAVAÇÃO DO FIRMWARE

O firmware FFBeast é gravado no STM32 com o STM32CubeProgrammer, normalmente por USB no modo DFU. Em situações de recuperação, pode ser utilizada a interface SWD com um programador ST-Link. O procedimento completo, incluindo drivers, entrada no modo de boot, seleção do arquivo e cuidados de alimentação, será detalhado no Manual de Montagem.

5.1 Fonte e resistor de frenagem

A fonte considerada para a etapa de potência possui tensão nominal de 36 V e potência de 1000 W. Sua corrente máxima teórica é obtida pela relação entre potência e tensão:

$$I = P / V = 1000 / 36 \approx 27,8 \text{ A}$$

Durante movimentos regenerativos, o motor pode devolver energia ao barramento de corrente contínua. Como uma fonte chaveada convencional normalmente não absorve essa energia, a controladora aciona um resistor de frenagem para convertê-la em calor e limitar a elevação da tensão do barramento.

Para um resistor de 4 Ω submetido a 36 V, a corrente e a potência instantâneas teóricas seriam:

$$I = V / R = 36 / 4 = 9 \text{ A}$$

$$P = V^2 / R = 36^2 / 4 = 324 \text{ W}$$

Como o resistor adotado possui potência nominal de 200 W, ele não pode dissipar 324 W de forma contínua. Sua utilização depende de acionamento intermitente, ciclo de trabalho reduzido, refrigeração adequada e configuração correta do circuito de frenagem. Além disso, durante a regeneração, a tensão do barramento pode ultrapassar 36 V, elevando ainda mais a potência instantânea. Portanto, esse cálculo é uma verificação inicial e não substitui o dimensionamento conforme os limites da revisão específica da controladora e as condições reais de uso.

ATENÇÃO TÉCNICA

O resistor deve ser instalado em local ventilado, afastado de cabos, plásticos e peças impressas em 3D. A temperatura, a tensão do barramento e os limites de corrente devem ser verificados durante os testes.

6. Motor BLDC de hoverboard

O motor utilizado no projeto é um motor BLDC proveniente de hoverboard. Ele possui ímãs permanentes no rotor e enrolamentos trifásicos no estator. Diferentemente de um motor de corrente contínua com escovas, sua comutação é realizada eletronicamente pela controladora.

O movimento ocorre quando a MKS energiza as três fases em uma sequência calculada, produzindo um campo magnético girante no estator. Esse campo interage com os ímãs do rotor e gera torque. Como motores de hoverboard possuem vários polos magnéticos e foram projetados para operar em baixas rotações, apresentam características adequadas para aplicações direct drive.

No cockpit, o motor é adaptado ao eixo do volante. O torque produzido depende principalmente da corrente das fases e da constante de torque do motor. Por isso, aumentar a corrente eleva a força percebida, mas também aumenta as perdas térmicas. Os limites devem ser configurados para evitar superaquecimento do motor, da controladora, dos cabos e da fonte.

O encoder instalado no eixo informa a posição angular. Com essa informação, a controladora determina a posição do rotor e ajusta as correntes necessárias. O motor pode então resistir ao movimento do usuário, reproduzir impactos, retornar ao centro e gerar efeitos associados à aderência, ao terreno e à dinâmica do veículo.

Quando recebe energia elétrica, o conjunto funciona como motor e produz torque. Quando é movimentado externamente pelo usuário, pode funcionar como gerador, devolvendo energia ao barramento. Essa energia regenerativa é direcionada ao resistor de frenagem.

A escolha do motor de hoverboard foi motivada pelo custo relativamente baixo, disponibilidade, alto torque em baixas rotações e compatibilidade com controladoras baseadas em ODrive. Essas características fazem dele uma alternativa recorrente em projetos artesanais de volantes direct drive.

7. Projeto de comunicação

A comunicação entre os componentes utiliza diferentes protocolos e interfaces, escolhidos de acordo com a função de cada módulo.

Ligação	Protocolo ou interface	Dados transportados
Arduino Leonardo ↔ computador	USB HID e USB CDC/Serial	Estados de botões, eixo do freio de mão e dados de telemetria.
SimHub → Arduino Leonardo	Comunicação serial virtual por USB	Velocidade, RPM, marcha, combustível, tempos, distância, sono e informações de volta.
Arduino Leonardo → LCD 20×4	I ² C	Caracteres, posição do cursor e comandos de atualização do display.
Jogo ↔ MKS/FFBeast	USB com dispositivo HID e efeitos de force feedback	Posição do volante, eixos, botões e efeitos de força.
Encoder → MKS	Sinais incrementais A e B; canal Z quando empregado	Posição, direção de movimento e referência de índice.
STM32 → etapa de potência	Sinais PWM trifásicos internos	Comandos de chaveamento dos MOSFETs para controle das fases do motor.

O Arduino Leonardo utiliza o ATmega32U4, que possui USB nativo. Essa característica permite apresentar simultaneamente um dispositivo HID ao sistema operacional e uma porta serial virtual CDC. Como HID, a placa envia os comandos físicos do cockpit. Como porta serial, recebe do SimHub as mensagens de telemetria.

A comunicação com o LCD ocorre pelo barramento I²C, utilizando as linhas SDA e SCL. O Arduino atua como controlador e o módulo do display como periférico. O endereço utilizado no projeto é 0x27, sujeito a confirmação conforme o adaptador instalado.

O multiplexador 74HC4067 e o contador CD4026 não empregam protocolos seriais. O primeiro é controlado por quatro linhas digitais de seleção de canal, enquanto o segundo recebe sinais digitais de clock e reset. Eles são tratados como interfaces lógicas auxiliares, e não como protocolos de comunicação.

No sistema de force feedback, o FFBeast recebe do jogo os efeitos de força e envia ao computador a posição do volante. Internamente, o STM32 transforma a referência de torque em comandos de controle das fases do motor. O aprofundamento sobre gravação, configuração e calibração da MKS será apresentado no Manual de Montagem.

8. Componentes associados ao Arduino Leonardo

O Arduino Leonardo foi escolhido como controlador dos comandos auxiliares devido ao ATmega32U4 com USB nativo. Com a biblioteca Joystick.h, a placa pode ser reconhecida como controle de jogos, enviando botões e eixos ao computador sem a necessidade de um conversor USB externo.

Componente	Aplicação no projeto	Motivo da escolha
------------	----------------------	-------------------

Arduino Leonardo	Controlador dos botões, câmbio, freio de mão e displays	USB nativo, compatibilidade com HID e quantidade de entradas adequada ao módulo auxiliar.
Botões e chave de seta	Comandos do veículo e funções do painel	Leitura digital simples e fácil associação a botões HID.
74HC4067	Expansão das entradas de botões e seleção das posições do câmbio	Permite ler até 16 canais utilizando uma única linha comum e quatro sinais de seleção.
Freio de mão analógico	Entrada proporcional enviada como eixo Rx	Permite ao jogo reconhecer diferentes níveis de acionamento.
CD4026	Controle do display de sete segmentos	Integra contador decimal e decodificador, reduzindo a quantidade de pinos necessários.
Display LCD I ² C 20×4	Apresentação da telemetria	Oferece quatro linhas e utiliza apenas SDA e SCL para comunicação.
Placa de interligação	Conexão organizada entre Arduino e periféricos	Reduz ligações soltas e facilita montagem, teste e manutenção.
Placa de ventoinhas	Acionamento do sistema de refrigeração	Auxilia na dissipação térmica do resistor, da MKS e do motor.

Os botões utilizam entradas digitais com resistores de pull-up internos quando aplicável. Essa configuração evita níveis indefinidos e reduz a quantidade de componentes externos. O freio de mão é lido por uma entrada analógica e enviado ao computador como eixo HID.

Os pedais não são lidos pelo Arduino Leonardo no arranjo atual. Eles são conectados às entradas analógicas da MKS e apresentados ao computador por meio do FFBeast Premium. Essa separação deve ser preservada na documentação e nos diagramas do projeto.

9. Software desenvolvido para o cockpit

Além dos firmwares executados no Arduino Leonardo, foi desenvolvido um software para computador com o objetivo de facilitar a seleção e a instalação dos perfis utilizados em cada jogo. A interface permite escolher entre Euro Truck Simulator 2 e DiRT Rally 2.0.

Cada jogo utiliza variáveis e layouts distintos. O perfil do Euro Truck Simulator 2 apresenta velocidade, RPM, marcha, combustível, descanso, horário e informações de navegação. O perfil do DiRT Rally 2.0 apresenta velocidade, RPM, volta atual, total de voltas e tempos de referência. Por esse motivo, foram mantidos códigos específicos para cada perfil.

Após a seleção do jogo, o software identifica o sketch correspondente e utiliza o Arduino CLI para compilar e gravar automaticamente o firmware no Arduino Leonardo. Dessa forma, o usuário não precisa abrir a Arduino IDE, localizar o arquivo, selecionar manualmente a placa e iniciar o upload.

A aplicação também identifica a porta associada ao Arduino e executa os comandos necessários à compilação e ao envio. O programa foi desenvolvido em Python, linguagem escolhida pela facilidade de construção de interfaces gráficas, execução de processos externos e manipulação de arquivos de configuração.

O Arduino CLI é uma ferramenta oficial de linha de comando que fornece recursos de detecção de placas, gerenciamento de plataformas e bibliotecas, compilação e upload. Ele não é um protocolo de comunicação. No sistema, funciona como uma ferramenta de automação acionada pela aplicação Python.

O SimHub continua responsável pela obtenção e transmissão da telemetria durante o jogo. O software em Python não substitui o SimHub; sua função é selecionar, organizar e gravar o firmware correspondente ao perfil desejado.

DOCUMENTAÇÃO DOS CÓDIGOS

Os códigos do Arduino, os scripts JavaScript do SimHub, a aplicação Python e os arquivos de configuração serão disponibilizados na pasta “Códigos”. Essa pasta conterá também um documento específico explicando bibliotecas, variáveis, funções, fluxo de execução, mensagens seriais e processo de compilação e gravação.

10. Modelagem de dados

A modelagem de dados descreve como as informações são obtidas, tratadas, armazenadas temporariamente em variáveis e transferidas entre os códigos. Como o projeto não utiliza banco de dados convencional, o foco está nos fluxos de informação e na estrutura das mensagens seriais.

10.1 Dados dos comandos físicos

O Arduino Leonardo lê os botões, as posições do câmbio e o freio de mão. Os estados dos canais do multiplexador e os estados anteriores dos botões são mantidos em vetores utilizados para detectar alterações e atualizar o dispositivo HID.

```
bool botoesMUX[16];
int estadoAntes[21];
```

A função de leitura seleciona cada canal do 74HC4067, registra o estado e o associa a um botão do joystick. O câmbio utiliza canais do multiplexador e também fornece o valor apresentado pelo display de sete segmentos. O freio de mão é lido por analogRead() e enviado ao eixo Rx.

```
Joystick.setRxAxis(analogRead(freioMao));
```

10.2 Dados recebidos do SimHub

Os scripts do SimHub coletam propriedades do jogo, verificam valores nulos ou inválidos, realizam conversões e montam uma mensagem textual. Os campos são separados pelo caractere ponto e vírgula e a mensagem termina com uma quebra de linha, utilizada pelo Arduino para identificar o fim do pacote.

10.3 Estrutura de dados do Euro Truck Simulator 2

O perfil do Euro Truck Simulator 2 utiliza 11 campos:

```
velocidade;rpm;combustível;marcha;diaAtual;horaAtual;dataEntrega;horaEntrega;tempoDestino;distância;sono
```

Pos.	Variável	Tipo no Arduino	Finalidade
1	Velocidade	int	Exibição em km/h.
2	RPM	int	Formação da barra de rotação.
3	Combustível	int	Formação da barra de combustível.
4	Marcha	String	Exibição da marcha.
5	Dia atual	String	Tela de informações.
6	Hora atual	String	Tela de informações.
7	Data de entrega	String	Campo reservado no formato atual.
8	Hora da entrega	String	Prazo restante da entrega.
9	Tempo até o destino	String	Tempo estimado de navegação.
10	Distância restante	String	Distância até o destino.

11	Sono	int	Formação da barra de descanso.
----	------	-----	--------------------------------

O script converte o tempo do jogo em dia da semana e horário, transforma segundos de navegação em horas e minutos e calcula o nível de cansaço a partir do tempo restante até o próximo descanso. Os textos são limitados para se adequarem ao LCD 20×4.

10.4 Estrutura de dados do DiRT Rally 2.0

O perfil do DiRT Rally 2.0 utiliza seis campos:

```
velocidade;rpm;voltaAtual;totalVoltas;melhorVoltaCorrida;melhorVoltaPiloto
```

Pos.	Variável	Tipo no Arduino	Finalidade
1	Velocidade	int	Exibição em km/h.
2	RPM	int	Formação da barra de rotação.
3	Volta atual	int	Indicação da volta em andamento.
4	Total de voltas	int	Quantidade de voltas da prova.
5	Melhor volta da corrida	String	Melhor tempo registrado na sessão.
6	Melhor volta do piloto	String	Tempo de referência pessoal.

A volta atual é calculada a partir do número de voltas concluídas, com correção para impedir que o valor ultrapasse o total da prova. Os tempos podem ser recebidos em segundos, milissegundos ou texto e são formatados como MM:SS.mmm.

10.5 Tratamento e atualização dos dados

No Arduino, a mensagem é recebida pela porta serial, separada pelo delimitador, convertida para tipos numéricos ou textuais e armazenada nas variáveis do perfil. Depois, as funções do LCD transformam percentuais em barras, organizam os textos e atualizam somente as regiões necessárias do display.

Os códigos mantêm valores anteriores para verificar se houve mudança antes de escrever novamente no LCD. Essa estratégia reduz atualizações desnecessárias e diminui a possibilidade de caracteres piscando ou alterações visuais causadas pela limpeza repetida da tela. Também é utilizado um intervalo entre atualizações para preservar a leitura contínua dos botões e sensores.

10.6 Diagrama geral do fluxo de dados

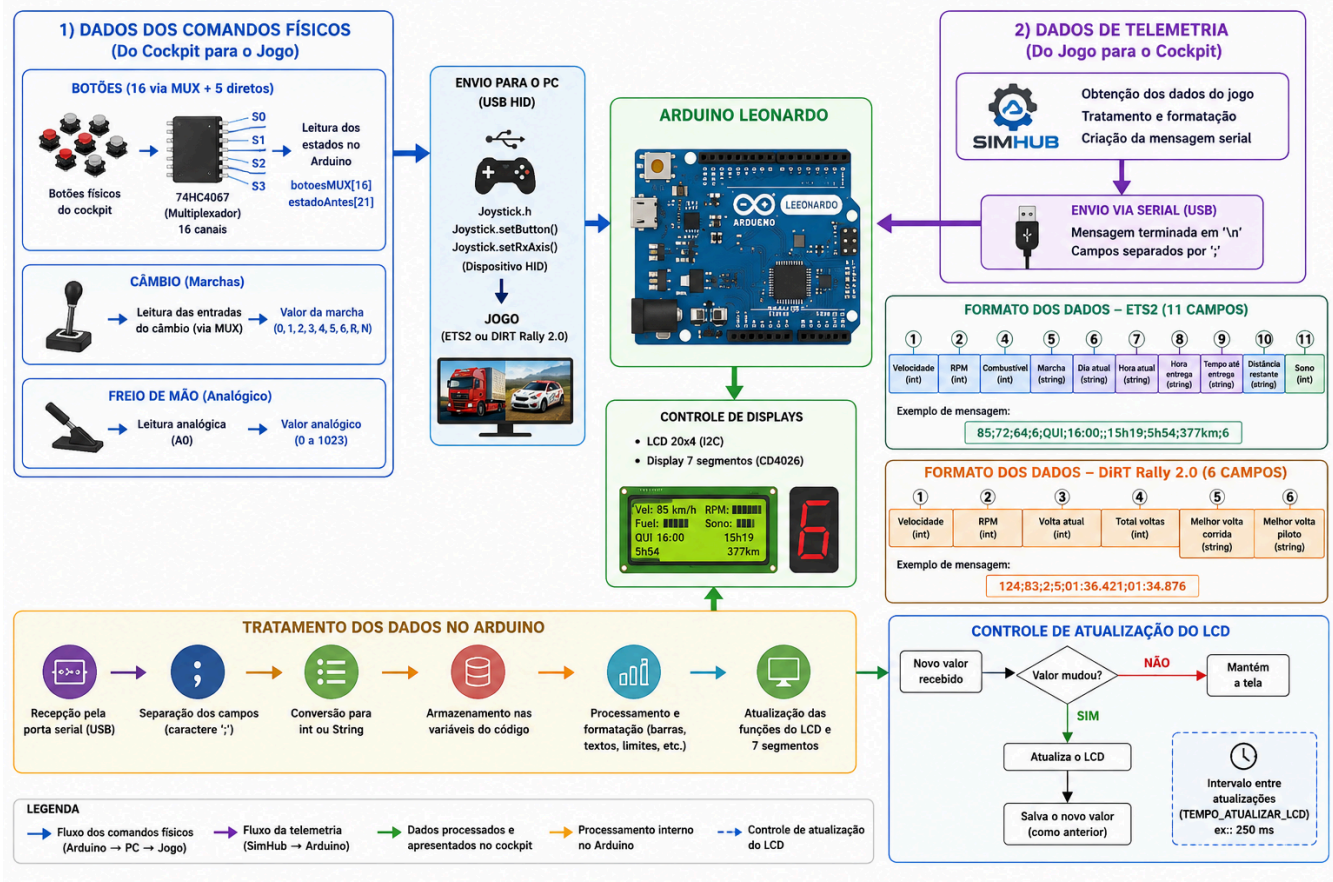


Figura 1 - Fluxo de comandos físicos, telemetria e atualização dos displays.

O diagrama resume os dois sentidos principais de comunicação: comandos físicos enviados do cockpit ao jogo e telemetria enviada do jogo ao painel. Ele também representa o tratamento interno das mensagens no Arduino e a atualização condicional do LCD.

11. Organização dos códigos

Os arquivos do projeto serão organizados em uma pasta específica denominada "Códigos", subdividida por função. Essa estrutura permitirá localizar facilmente os firmwares, scripts e ferramentas utilizados no sistema.

Pasta sugerida	Conteúdo
Códigos/Arduino/ETS2	Sketch do Arduino Leonardo para Euro Truck Simulator 2.
Códigos/Arduino/DiRT Rally2	Sketch do Arduino Leonardo para DiRT Rally 2.0.
Códigos/SimHub	Scripts JavaScript responsáveis pela coleta, tratamento e envio da telemetria.
Códigos/Software-Python	Código-fonte da interface de seleção e gravação dos perfis.
Códigos/Configurações	Arquivos JSON, caminhos e parâmetros utilizados pela aplicação.
Códigos/Documentação	Documento que explica a estrutura e o funcionamento dos códigos.

A documentação dos códigos deverá descrever as bibliotecas utilizadas, a atribuição dos pinos, as funções de leitura dos controles, o formato das mensagens, a atualização dos displays, a detecção do Arduino, os comandos do Arduino CLI e o tratamento de erros.

12. Considerações de segurança e limitações

- A etapa de potência trabalha com correntes elevadas. Fusíveis, bitola dos cabos, conectores, aterramento e proteção mecânica devem ser dimensionados adequadamente.
- A fonte de 1000 W indica capacidade máxima de fornecimento, não o consumo contínuo do sistema.
- O resistor de frenagem pode atingir temperaturas perigosas e deve possuir dissipação e ventilação apropriadas.
- Os limites de corrente do motor e da controladora devem ser definidos progressivamente durante os testes.
- A compatibilidade e os limites elétricos devem ser confirmados para a revisão exata da placa MKS utilizada.
- O botão de emergência e uma forma rápida de interrupção da alimentação da etapa de potência são recomendados.
- As ventoinhas auxiliam na refrigeração, mas não substituem a medição de temperatura e a instalação adequada de dissipadores.
- A calibração do encoder, dos pedais e do freio de mão deve ser realizada antes do uso em força elevada.

13. Considerações finais

O cockpit simulador reúne diferentes áreas da Engenharia de Computação em um único sistema: programação embarcada, software para computador, comunicação, leitura de sensores, interfaces homem-máquina, eletrônica de potência e controle de motores.

A divisão modular do projeto permite que o volante direct drive, os controles auxiliares, a telemetria e o software de seleção de perfis sejam desenvolvidos e testados separadamente. Ao mesmo tempo, a integração desses módulos resulta em um equipamento funcional, interativo e adequado a demonstrações acadêmicas.

A documentação organizada em projeto elétrico, projeto físico, comunicação, modelagem de software, modelagem de dados, códigos e manuais permitirá registrar as decisões adotadas e facilitar futuras correções e expansões. O projeto demonstra que conhecimentos acadêmicos podem ser empregados na construção de soluções tecnológicas voltadas tanto à aprendizagem quanto ao lazer.

Durante o desenvolvimento do projeto, alguns problemas e comportamentos inesperados foram observados. Um deles está relacionado à elevada força produzida pelo motor de hoverboard. Em determinadas situações, caso os limites de força e corrente não estejam configurados corretamente, o volante pode apresentar movimentos bruscos e até perder a referência de posição do eixo. Esse comportamento pode estar associado tanto a uma configuração inadequada no software FFBeast quanto à calibração do encoder e dos parâmetros de controle do motor.

Outro ponto identificado foi o pequeno atraso na leitura das marchas conectadas ao multiplexador. Embora esse atraso não comprometa significativamente o funcionamento na maioria dos jogos, trocas de marcha muito rápidas podem não ser reconhecidas corretamente. Isso pode ocorrer, por exemplo, quando o usuário movimenta a alavanca e solta a embreagem antes que o Arduino conclua a seleção do canal e a leitura digital correspondente. Mesmo com essa limitação, o sistema continua funcional, mas pode exigir ajustes no código, no intervalo de varredura do multiplexador ou na forma de tratamento das entradas para aumentar a confiabilidade.

Referências técnicas

- [1] ARDUINO. Arduino Leonardo: placa baseada no ATmega32U4 com comunicação USB nativa e porta serial virtual CDC.
- [2] ARDUINO. Arduino CLI: ferramenta oficial para detecção de placas, compilação e upload de sketches.
- [3] ODRIVE ROBOTICS. Hardware Configuration: controle da tensão do barramento e uso de resistor de frenagem.
- [4] ODRIVE ROBOTICS. Brake Resistor Selection: critérios elétricos para seleção e limitação de corrente do resistor.
- [5] FFBEAST. Documentação do projeto: hardware compatível, instalação do firmware, configuração e licenciamento.
- [6] STMICROELECTRONICS. STM32CubeProgrammer: programação de microcontroladores STM32 por DFU e interfaces de depuração.
- [7] GUSTAVO G-TEC. Conteúdo educacional sobre construção de volantes e controles artesanais para simuladores.